

Gerald Stanford
PID CONTROLLER

PART 1:
Determining the Step Response of the numerator:

```

clear; close all; clc;

% Define symbolic variable in the context of transfer function
s = tf("s");

%sys = 20 / (s + 4)
%sys = (s + 4) / ( (s + 2)*(s+9) )

sys = 3 / ((s+0.1) * (s + 0.5))
% Compute poles and zeros
poles = pole(sys);
disp('The poles are:');
for i = 1:length(poles)
    fprintf('Pole %d = %s\n', i, num2str(poles(i), 4));
end

zeros = zero(sys);
disp('The zeros are:');
for i = 1:length(zeros)
    fprintf('Zero %d = %2.3f\n', i, zeros(i));
end

% Plot poles and zeros
figure('Name', 'Overdamped system', 'Units', 'normalized', 'Position', [0.1 0.1 0.6 0.45]);

ax1 = subplot(1, 2, 1);
hold on
scatter(real(poles), imag(poles), 100, 'Marker', 'x', 'MarkerEdgeColor', 'r');
scatter(real(zeros), imag(zeros), 100, 'Marker', 'o', 'MarkerEdgeColor', 'b');
xline(0, 'k')
yline(0, 'k')
legend({'Poles', 'Zeros'})
title('Poles and Zeros Plot');

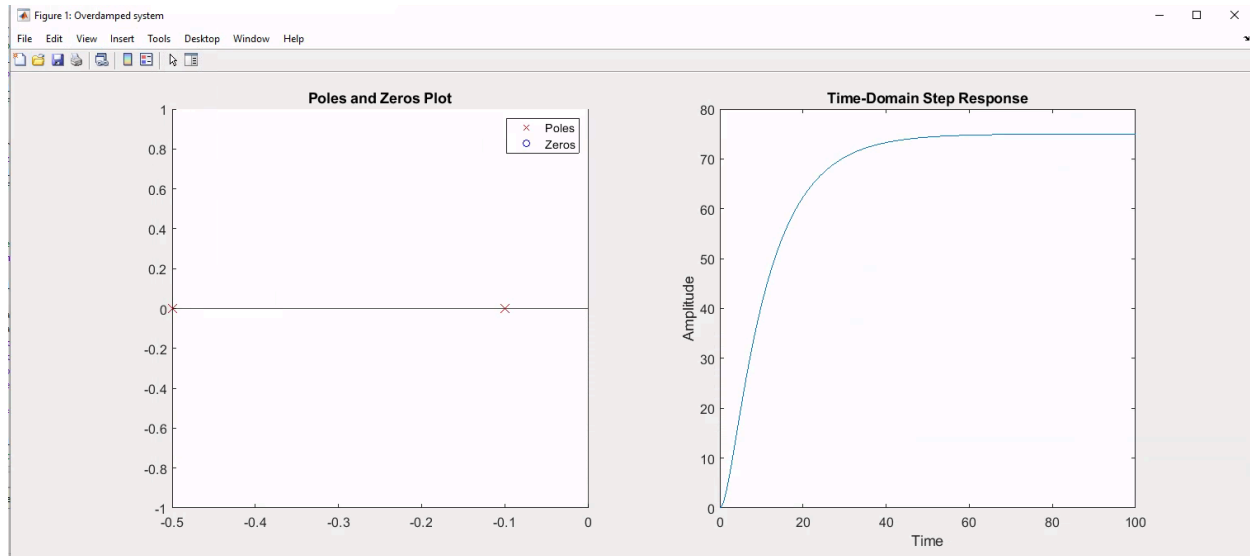
set(ax1, 'FontSize', 12)

timeEnd = 100
% Compute time-domain response
t = linspace(0, timeEnd, 1000); % adjust time vector as needed
c = step(sys, t);

% Plot time-domain response
ax2 = subplot(1, 2, 2);
plot(t, c * 1.25);
title('Time-Domain Step Response');
xlabel('Time');
ylabel('Amplitude');

set(ax2, 'FontSize', 12)

```



2

high gain about 75

2 poles lhp, both on real axis

Overdamped curve

Stable system

Settling time about 41secs

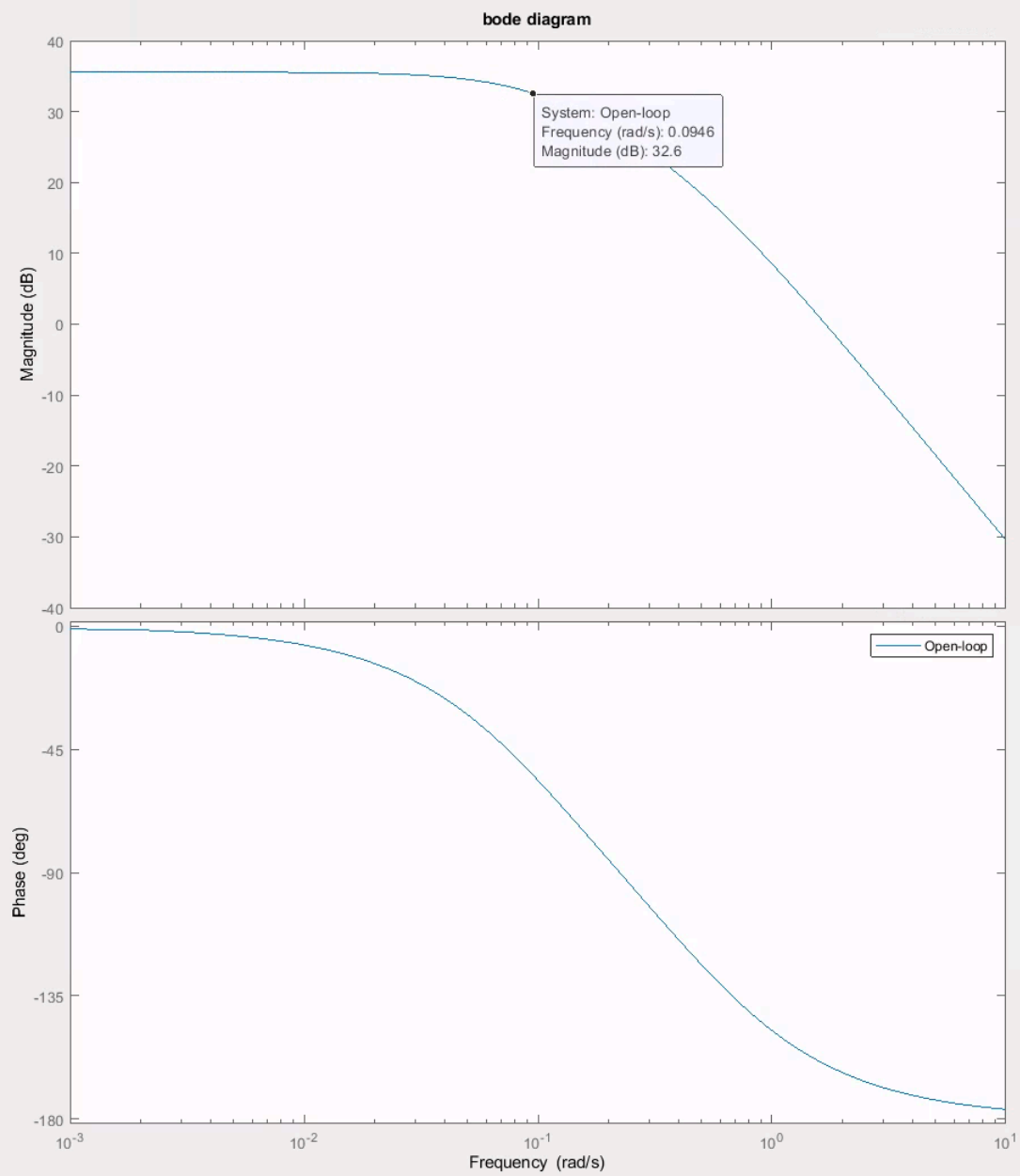
3.

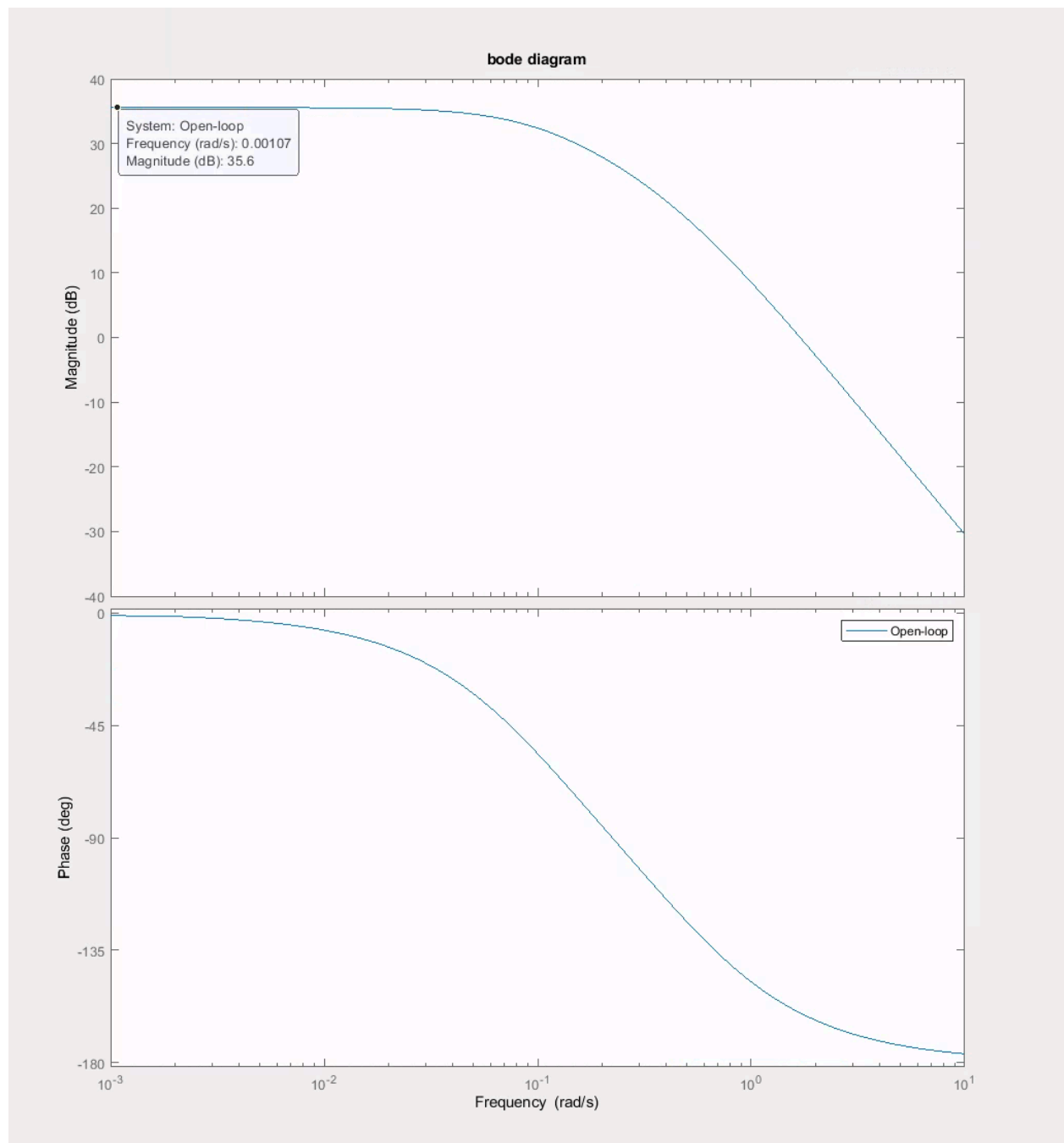
```
% Bode diagram

% symbolic variable `s`
s = tf('s');

% Open-loop system
%          s + 10
%  G(s) = -----
%          (s + 1)^2 (s + 100)
sys = 3 / ((s+0.1) * (s + 0.5))

% diagram
bode(sys);
title("bode diagram")
legend(["Open-loop", "Closed-loop"], "Location", "northeast");
```





Shows the initial frequency and the corner freq

4.

```

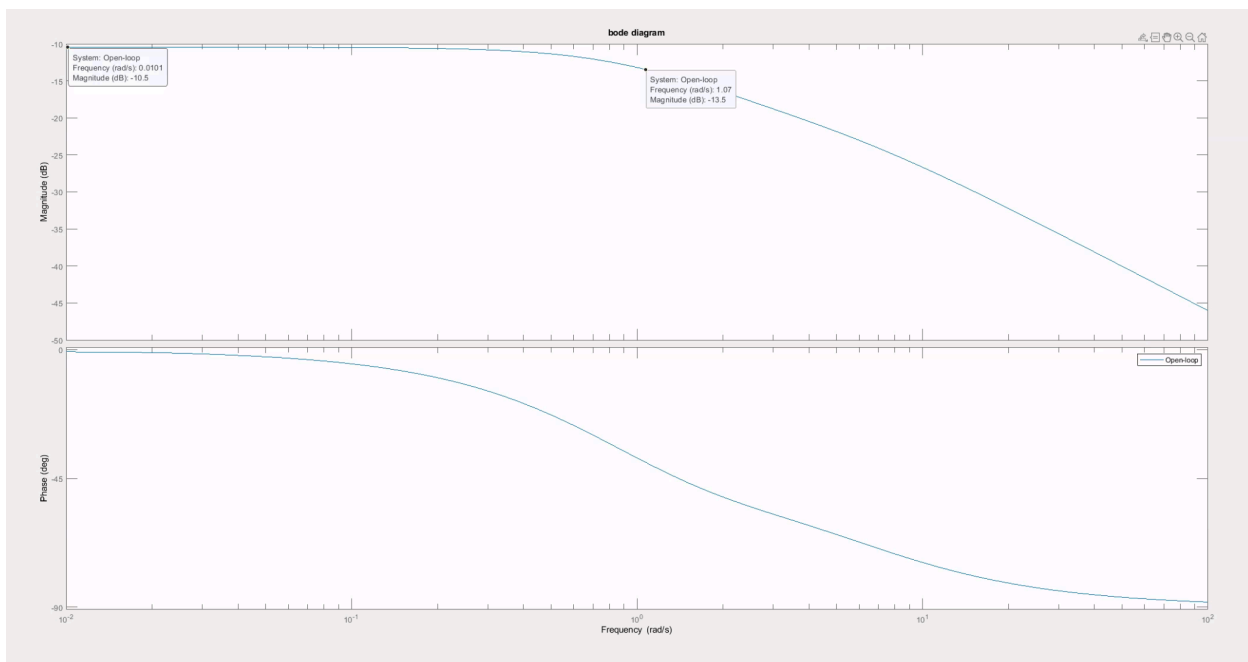
% Bode diagram

% symbolic variable `s`
s = tf('s');

% Open-loop system
%          s + 10
% G(s) = -----
%      (s + 1)^2 (s + 100)
sys = .5*(s+3) / ((s+1) * (s + 5))

% diagram
bode(sys);
title("bode diagram")
legend(["Open-loop", "Closed-loop"], "Location", "northeast");

```



Increased 3db freq by adding a zero, decreasing gain and moving poles farther left.

5.

Closed loop code.

```

1 clear; close all; clc;
2
3 % Define symbolic variable in the context of transfer function
4 s = tf("s");
5
6 %sys = 1 / (s + 4)
7 %sys = (s + 4) / ( (s + 2)*(s+9) )
8
9 sys = 3 / ((s+0.1) * (s + 0.5))
10 % Compute poles and zeros
11 poles = pole(sys);
12 disp('The poles are:');
13 for i = 1:length(poles)
14     fprintf('Pole %d = %s\n', i, num2str(poles(i), 4));
15 end
16
17 zeros = zero(sys);
18 disp('The zeros are:');
19 for i = 1:length(zeros)
20     fprintf('Zero %d = %2.3f\n', i, zeros(i));
21 end
22
23
24 % Plot poles and zeros
25 figure('Name', 'Overdamped system', 'Units', 'normalized', 'Position', [0.1 0.1 0.6 0.45]);
26
27 ax1 = subplot(1, 2, 1);
28 hold on
29 scatter(real(poles), imag(poles), 100, 'Marker', 'x', 'MarkerEdgeColor', 'r');
30 scatter(real(zeros), imag(zeros), 100, 'Marker', 'o', 'MarkerEdgeColor', 'b');
31 xline(0, 'k')
32 yline(0, 'k')
33 legend({'Poles', 'Zeros'})
34 title('Poles and Zeros Plot');
35
36 set(ax1, 'FontSize', 12)
37
38 timeEnd = 100
39 % Compute time-domain response
40 t = linspace(0, timeEnd, 1000); % adjust time vector as needed
41 c = step(sys, t);
42
43 % Plot time-domain response
44 ax2 = subplot(1, 2, 2);
45 plot(t, c * 1.25);
46 title('Time-Domain Step Response');
47 xlabel('Time');
48 ylabel('Amplitude');
49
50 set(ax2, 'FontSize', 12);

```

Command Window

New to MATLAB? See resources for [Getting Started](#).

Continuous-time transfer function.

[Model Properties](#)

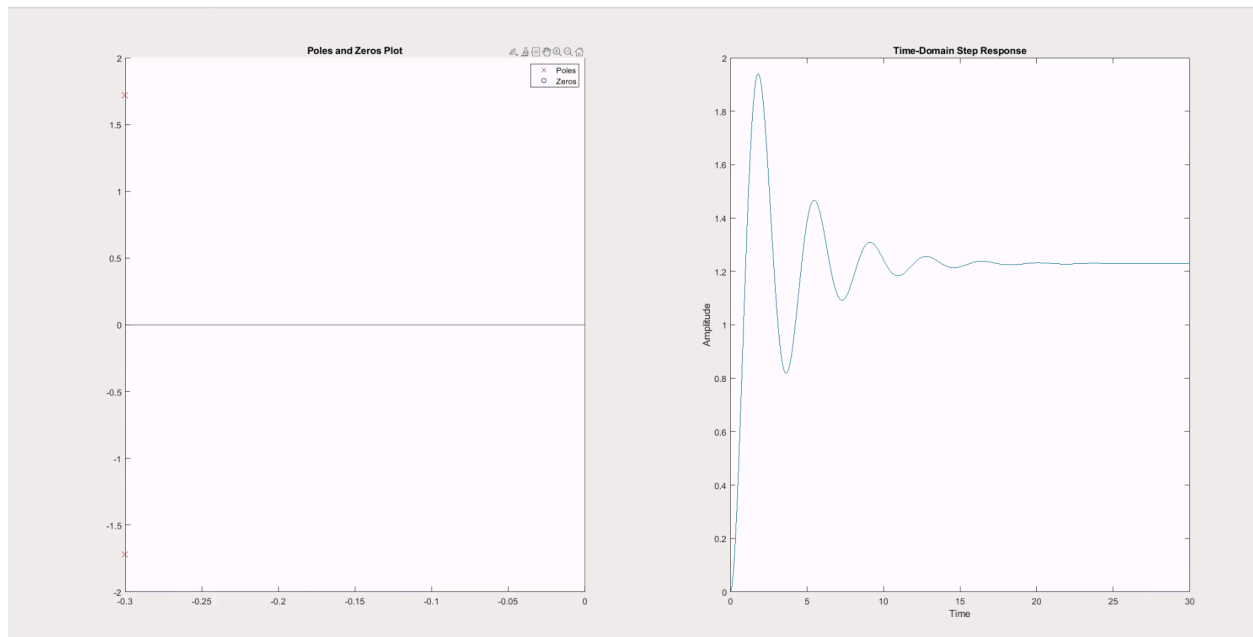
The poles are:

Pole 1 = -0.3+1.72i

Pole 2 = -0.3-1.72i

The zeros are:

Closed loop response



Differences:

The poles changed positions and no longer on the real axis

More oscillations in signal

Was overdamped now underdamped with overshoot

Gain is significantly lower

Settling time is shorter

PART 2:

Was trying to test out different values of K_p and found that at a certain point only the imaginary component of the poles would change. I believe this is the breakaway point

So after trying to figure out the portional gain and having no luck with the the derivative gain, tried the Ziegler-Nichols method, but the results were confusing.

```

clear; close all; clc;
%1.750
% Define symbolic variable in the context of transfer function
s = tf('s');
kp = 1.75
kd = 0
ki = 0
controller = pid(kp,ki,kd)
sys = 3 / ((s+0.1) * (s + 0.5))

%open loop
step(sys)

pause()

%closed loop
feedback_sys = feedback(sys, 1);
step(feedback_sys)

% step data
s_info = stepinfo(feedback_sys);

overshoot = s_info.Overshoot;
settling_time = s_info.SettlingTime;

fprintf('Proportional gain (kp): %2.3f\n', kp)
fprintf('Percentage overshoot: %2.2f%%\n', overshoot)
fprintf('Settling time: %2.2f s\n\n', settling_time)

pause()

% Root-locus plot
if kp == 0
    rlocus(sys)
    axis([-22 3 -15 15])

    sgrid

    [kp, poles] = rlocfind(sys);

% feedback system with unit gain
sys_cl = feedback(kp * sys, 1);

% get the step response
figure
step(sys_cl)
s_info = stepinfo(sys_cl);

overshoot = s_info.Overshoot;
settling_time = s_info.SettlingTime;

fprintf('Proportional gain (kp): %2.3f\n', kp)
fprintf('Percentage overshoot: %2.2f%%\n', overshoot)
fprintf('Settling time: %2.2f s\n\n', settling_time)

pause()
end

```

```

%find kd
for kd = 0:10
    controller = pid(kp,ki,kd)
    sys = 3 / ((s+0.1) * (s + 0.5))
    feedback_sys = feedback(controller * sys, 1);
    %% step(feedback_sys)

    % step data
    s_info = stepinfo(feedback_sys);

    overshoot = s_info.Overshoot;
    settling_time = s_info.SettlingTime;

    fprintf('Proportional gain (kp): %2.3f\n', kp)
    fprintf('Derivative gain (kd): %2.3f\n', kd)
    fprintf('Percentage overshoot: %2.2f%%\n', overshoot)
    fprintf('Settling time: %2.2f s\n\n', settling_time)

    if settling_time <= 3 & overshoot <= 5
        break
    end
end
end

```

```

%output latest
step(feedback_sys)

% step data
s_info = stepinfo(feedback_sys);

overshoot = s_info.Overshoot;
settling_time = s_info.SettlingTime;

fprintf('Proportional gain (kp): %2.3f\n', kp)
fprintf('Percentage overshoot: %2.2f%%\n', overshoot)
fprintf('Settling time: %2.2f s\n\n', settling_time)

pause()

% final plot
kp = 1.75
kd = 2
ki = 0
controller = pid(kp,ki,kd)
sys = 3 / ((s+0.1) * (s + 0.5))

%closed loop PID
fprintf('Final Ouput \n')

feedback_sys = feedback(controller * sys, 1);
step(feedback_sys)

s_info = stepinfo(feedback_sys);
overshoot = s_info.Overshoot;
settling_time = s_info.SettlingTime;

fprintf('Proportional gain (kp): %2.3f\n', kp)
fprintf('Percentage overshoot: %2.2f%%\n', overshoot)
fprintf('Settling time: %2.2f s\n\n', settling_time)

pause()

```

Final Results:

Continuous-time transfer function.

Model Properties

Proportional gain (kp): 1.750

Derivative gain (kd): 2.000

Percentage overshoot: 2.92%

Settling time: 1.51 s

Proportional gain (kp): 1.750

Percentage overshoot: 2.92%

Settling time: 1.51 s

f_x

